Real-Time Flight Delay Predictor

Complete Code Walkthrough — All Files Explained

SJSU DATA-228 · Spring 2026 · Anees Saheba Guddi

End-to-End Pipeline

```
BTS CSV Data (raw airline on-time performance files)
↓
[1] Ingestion → ingest_bts_to_hdfs.py → HDFS Parquet
↓
[2] Training → train_local.py → LR + GBT PipelineModels
↓
[3a] Batch → batch_inference.py → HDFS batch_predictions/
↓ ■
[3b] Producer → kafka_producer.py → Kafka topic: flight-events
↓
[3c] Consumer → streaming_consumer.py → HDFS streaming_predictions/
↓ ■
[4] Benchmark → benchmark.py → benchmark_report.json
```

Files Covered in This Document

#	File	Owner	Purpose
1	docker-compose.yml	Rish	7-service infrastructure
2	scripts/setup_hdfs.sh	Kartheek	Create HDFS directories
3	scripts/run_pipeline.sh	Rish	Orchestrate full pipeline (7 steps)
4	src/ingestion/ingest_bts_to_hdfs.py	Kartheek	CSV → HDFS Parquet
5	src/training/train_local.py	Manjot	LR + GBT training & CV
6	src/streaming/kafka_producer.py	Keon	Replay BTS data to Kafka
7	src/streaming/streaming_consumer.py	Keon	Real-time Spark inference
8	src/batch/batch_inference.py	Kartheek	Static batch scoring

9	src/evaluation/benchmark.py	Anees	Compare batch vs. streaming
10	models/benchmark_report.json	Anees	Final results output

SECTION 1

docker-compose.yml — Infrastructure

docker-compose.yml — Infrastructure · Owner: Rish

Defines every service the system needs as Docker containers connected on a shared bridge network called **flight-net**. Running docker compose up -d starts the entire infrastructure in dependency order.

Services at a Glance

kafka	Apache Kafka 3.7 in KRaft mode (no Zookeeper). Ports 9092 (internal) and 9093 (external/host). Auto-creates topics, 24-hour log retention.
kafka-init	One-shot container that runs after Kafka is healthy and creates the 'flight-events' topic with 4 partitions, replication factor 1.
hdfs-namenode	Hadoop 3.2.1 namenode. Exposes web UI on port 9870 and RPC on port 9000. Stores filesystem metadata on a named Docker volume.
hdfs-datanode	Hadoop datanode. Stores actual block data. Starts only after namenode is healthy.
spark-master	Spark 3.5 standalone master. Web UI port 8080, cluster RPC port 7077. Mounts ./src and ./data as read-only volumes.
spark-worker-1/2	Two workers, each with 4 cores and 4 GB RAM, registered to spark-master:7077.

Key design choices:

- KRaft mode removes the Zookeeper dependency, simplifying the stack from 8 services to 7.
- All services have **healthchecks** so Docker waits for real readiness before starting dependents.
- HDFS volumes are named (not bind-mounts), so data survives container restarts.
- Kafka message max is 10 MB to handle large JSON flight batches.

Listener configuration:

```
PLAINTEXT://kafka:9092 ← Spark / consumer talks to Kafka inside Docker
EXTERNAL://localhost:9093 ← kafka_producer.py on your laptop talks to Kafka
```

SECTION 2

scripts/setup_hdfs.sh — HDFS Directory Setup

scripts/setup_hdfs.sh Script · Owner: Kartheek

A short shell script that runs Hadoop CLI commands inside the namenode container to create the HDFS directory tree before any data is written.

Directories created:

```
hdfs://hdfs-namenode:9000/data/flights ← raw Parquet ingestion target
hdfs://hdfs-namenode:9000/models ← trained ML pipeline artifacts
hdfs://hdfs-namenode:9000/output/batch_predictions
hdfs://hdfs-namenode:9000/output/streaming_predictions
hdfs://hdfs-namenode:9000/checkpoints/streaming ← Spark checkpoint dir
```

Uses hdfs dfs -mkdir -p (idempotent — safe to rerun). Sets permissions to 777 so Spark workers can write without credential issues.

SECTION 3

scripts/run_pipeline.sh — Full Pipeline Orchestration

scripts/run_pipeline.sh Script · Owner: Rish

The master orchestration script. It runs all 7 pipeline steps in sequence, logs everything to ./logs/pipeline_TIMESTAMP.log, and can skip any step via flags. All spark-submit calls are parameterised via environment variables with sensible defaults.

7 steps in order:

Step 1	`setup_hdfs()`	Runs setup_hdfs.sh to create HDFS directories.
Step 2	`ingest_data()`	spark-submit ingest_bts_to_hdfs.py for years 2018–2024.
Step 3	`train_models()`	spark-submit train_model.py with 5-fold CV on cluster.
Step 4	`start_streaming_consumer()`	spark-submit streaming_consumer.py in background; waits 30s for init.
Step 5	`run_kafka_producer()`	python3 kafka_producer.py streams 2024 data; waits 60s after; kills consumer.
Step 6	`run_batch_inference()`	spark-submit batch_inference.py on 2024 test data.
Step 7	`run_benchmark()`	spark-submit benchmark.py; writes benchmark_report.json.

CLI flags:

```
bash scripts/run_pipeline.sh --skip-ingest --skip-training
# Useful when models are already trained — jumps straight to streaming + benchmark
```

An EXIT trap calls cleanup() which kills the background consumer PID on any failure, preventing orphan Spark jobs.

SECTION 4

src/ingestion/ingest_bts_to_hdfs.py — Data Ingestion

src/ingestion/ingest_bts_to_hdfs.py Data · Owner: Kartheek

Reads raw BTS Airline On-Time Performance CSV files (2018–2024, ~35 GB compressed), cleans them, creates the binary label column, and writes Parquet to HDFS partitioned by YEAR and MONTH.

Step-by-step inside main():

- **resolve_csv_paths()** — supports both year-subdirectory layouts and flat directories.
- **read_raw_csv()** — reads all CSVs as strings initially (inferSchema=false avoids type mismatches). Renames mixed-case BTS columns to standard uppercase names via BTS_COLUMN_MAP.
- **clean_and_transform()** — drops rows where ARR_DELAY is null (cancelled/diverted flights), fills delay breakdown columns with 0 for on-time flights, drops rows missing critical features.
- **Label creation** — label = 1 if ARR_DELAY > 15 else 0. Binary classification.
- **write_to_hdfs()** — writes Parquet partitioned by YEAR and MONTH for efficient year-filtered reads downstream. Snappy compression.

Column rename map (sample):

```
BTS_COLUMN_MAP = {
"Reporting_Airline": "OP_UNIQUE_CARRIER",
"CRSDepTime": "CRS_DEP_TIME",
"ArrDelay": "ARR_DELAY",
"DayOfWeek": "DAY_OF_WEEK",
... (18 mappings total)
}
```

Partitioning strategy:

Repartitions to ~500k rows per Parquet file using repartition(n, 'YEAR', 'MONTH'). When batch_inference.py later filters to YEAR=2024, Spark uses partition pruning to skip all other years — dramatically reducing I/O on 35 GB of data.

SECTION 5

src/training/train_local.py — ML Model Training

src/training/train_local.py ML · Owner: Manjot

The core ML script. Trains two Spark MLlib models — Logistic Regression (baseline) and Gradient Boosted Trees (primary) — inside full Spark ML Pipelines with cross-validation. Runs in local[*] mode so no cluster is needed for development.

Features used:

Categorical (3)	OP_UNIQUE_CARRIER, ORIGIN, DEST — airline carrier code, origin airport, destination airport.
Numeric (6)	DAY_OF_WEEK, CRS_DEP_TIME, DEP_DELAY, CRS_ELAPSED_TIME, DISTANCE, MONTH.
Label	ARR_DELAY > 15 min → 1 (delayed), else 0 (on-time).

Shared preprocessing pipeline (4 stages):

```
Stage 1: StringIndexer (×3) — encodes carrier/origin/dest to numeric index
Stage 2: Imputer — fills missing numeric values with median
Stage 3: VectorAssembler — combines all features → raw_features vector
Stage 4: StandardScaler — normalizes (withMean=True, withStd=True)
(critical for LR; harmless for GBT)
```

Class weighting:

About 77% of flights are on-time, only 23% are delayed — an imbalanced dataset. The script computes weight = total / (n_classes × class_count) for each class and assigns it via a class_weight column, so the model penalises missed delays more heavily than incorrect on-time predictions.

Cross-validation grid:

Model	Hyperparameter	Values Searched	Combos
GBT	maxDepth / maxIter / stepSize	{4,5} × {30,50} × {0.05,0.1}	8 × 3 folds = 24 jobs
LR	regParam / elasticNetParam	{0.001,0.01,0.1} × {0.0,0.5}	6 × 3 folds = 18 jobs

Model serialization:

Both models are saved as complete **PipelineModel** objects (model.write().overwrite().save(path)). This means the fitted StringIndexer vocabularies, imputer medians, scaler statistics, AND model weights are all bundled together. When the streaming consumer loads the pipeline later, it applies the exact same transformations — preventing training/serving feature skew.

Achieved results (Dec 2021 BTS data):

Model	AUC-ROC	AUC-PR	F1	Precision	Recall
GBTClassifier	0.9341	—	0.9018	0.9022	0.9015
LogisticRegression	~0.85	—	~0.82	~0.83	~0.82

GBT F1=0.9018 far exceeds the mid-term target of ≥ 0.70.

SECTION 6

src/streaming/kafka_producer.py — Kafka Producer

src/streaming/kafka_producer.py Streaming · Owner: Keon

Replays 2024 BTS CSV records as a live JSON stream to Kafka, simulating real-time flight event ingestion. Rate-limited to a configurable messages/second target.

Key functions:

- **build_producer()** — connects with acks='all' (durable), gzip compression, 64 KB batches, 10 ms linger for throughput. Retries 5× if Kafka is not yet ready.
- **discover_csv_files()** — walks the input directory recursively for all .csv files, sorted by name to ensure deterministic replay order.
- **row_to_dict()** — cleans each CSV row, casts numeric columns to float (None if empty/NA), and adds producer_ts = time.time(). This timestamp is the start of the latency clock.
- **produce_records()** — token-bucket rate limiter: computes interval = 1/rate seconds, sleeps for the remaining time in each cycle after send. Uses ORIGIN as the Kafka partition key.

Partition key rationale:

Using the departure airport (ORIGIN) as the Kafka partition key means all flights from the same origin are always routed to the same partition. This preserves temporal ordering per origin, which matters for any analysis that groups flights by departure airport.

Rate limiting:

```
interval = 1.0 / rate # e.g. 100 msg/s → interval = 0.01s
for record in records:
send_start = time.time()
producer.send(topic, key=origin, value=record)
elapsed = time.time() - send_start
if interval - elapsed > 0:
time.sleep(interval - elapsed) # sleep only remaining time
```

CLI usage:

```
python src/streaming/kafka_producer.py \
--input-path data/raw/2024 \
--kafka-bootstrap localhost:9093 \
--topic flight-events \
--rate 500 \
--loop # replay indefinitely for long-running tests
```

SECTION 7

src/streaming/streaming_consumer.py — Spark Streaming Consumer

src/streaming/streaming_consumer.py Streaming · Owner: Keon

A Spark Structured Streaming application that subscribes to the Kafka topic, deserializes JSON flight events, runs them through the GBT PipelineModel, measures end-to-end latency, and writes predictions to HDFS every 10 seconds.

Data flow inside the job:

```
Kafka topic 'flight-events'
→ read_kafka_stream() # subscribe, max 10k offsets per trigger
→ parse_kafka_messages() # from_json() with explicit FLIGHT_EVENT_SCHEMA
→ foreachBatch handler every 10 seconds:
model.transform(batch_df) # GBT pipeline inference
latency_ms = (consumer_ts - producer_ts) * 1000
output_df.write.parquet(output_path, mode='append')
log latency stats (mean, p50, p95, p99)
```

Why explicit schema (not inferSchema)?

Spark's JSON schema inference requires reading a sample of the data first, adding latency to stream startup. An explicit **StructType** schema makes parsing deterministic and faster. It also fails clearly if the producer sends an unexpected field instead of silently dropping it.

Back-pressure:

maxOffsetsPerTrigger=10_000 caps Kafka reads per micro-batch. If the producer bursts to 50,000 messages, Spark processes them across 5 batches rather than all at once, keeping per-batch memory bounded.

Latency formula:

```
# producer_ts: epoch seconds stamped by kafka_producer.py at send time
# consumer_ts: epoch seconds at the start of this micro-batch
latency_ms = (consumer_ts - producer_ts) * 1000
# Per the benchmark report, observed latency on Dec 2021 data:
# mean = 5,303 ms | p50 = 5,212 ms | p99 = 10,438 ms
```

Output columns written to HDFS:

```
batch_id, kafka_timestamp, kafka_partition, kafka_offset,
YEAR, MONTH, DAY_OF_MONTH, DAY_OF_WEEK, OP_UNIQUE_CARRIER,
ORIGIN, DEST, CRS_DEP_TIME, DEP_DELAY, ARR_DELAY, DISTANCE,
prediction, probability, rawPrediction,
producer_ts, consumer_ts, latency_ms
```

SECTION 8

src/batch/batch_inference.py — Batch Inference

src/batch/batch_inference.py Batch · Owner: Kartheek

The **control group** in the benchmark. Loads the 2024 test year from HDFS, applies the same saved GBT PipelineModel, evaluates classification metrics, and writes predictions to HDFS. No Kafka involved — pure batch Spark job.

Step-by-step:

- **load_test_data()** — reads HDFS Parquet, filters to YEAR=2024 using partition pruning (fast). Casts numeric columns to DoubleType for the pipeline.
- **load_model()** — PipelineModel.load() from HDFS. This is the same artifact training saved.
- **run_inference()** — model.transform() followed by .cache() and .count() to force Spark to materialize the lazy computation and time it accurately.
- **evaluate_predictions()** — computes AUC-ROC, AUC-PR, F1, precision, recall, accuracy, AND a full confusion matrix (TP/TN/FP/FN).
- **write_predictions()** — appends output Parquet to HDFS. The benchmark script later reads this.

Confusion matrix computation:

```
tp = df.filter((prediction==1) & (label==1)).count()
tn = df.filter((prediction==0) & (label==0)).count()
fp = df.filter((prediction==1) & (label==0)).count() # false alarm
fn = df.filter((prediction==0) & (label==1)).count() # missed delay
```

Batch results (537,183 records from Dec 2021 BTS data):

Metric	Batch Value
AUC-ROC	0.9386
AUC-PR	0.8877
F1	0.9029
Precision	0.9058
Recall	0.9011
Accuracy	0.9011
% Delayed	23.7%

SECTION 9

src/evaluation/benchmark.py — Benchmark & Evaluation

src/evaluation/benchmark.py — Evaluation · Owner: Anees

The evaluation module — your code. Loads both batch and streaming prediction outputs, computes classification metrics, latency and throughput statistics, applies Differential Privacy noise for safe public reporting, and runs an LSH-based anomaly detection pass. Writes everything to benchmark_report.json.

Technique 1 — Differential Privacy (Laplace Mechanism)

After computing true aggregate metrics (F1, AUC-ROC, accuracy), the script adds Laplace noise before writing the public-facing report. This ensures that no individual flight record's true delay label can be inferred from published statistics.

```
# Laplace mechanism: noise ~ Laplace(0, sensitivity / epsilon)
# sensitivity = 0.01 (one record changes accuracy by at most 1/n)
# epsilon = 1.0 (moderate privacy budget)
# noise scale = 0.01 / 1.0 = 0.01
def dp_metric(value, epsilon=1.0, sensitivity=0.01):
mech = diffprivlib.Laplace(epsilon=epsilon, sensitivity=sensitivity)
return clip(mech.randomise(value), 0.0, 1.0)
# Example: true F1=0.9029 → published as F1≈0.911 (noisy but close)
```

Technique 2 — LSH Anomaly Detection (datasketch MinHashLSH)

After scoring, the script builds MinHash signatures for 5,000 streaming predictions using categorical features (carrier + origin + dest). It inserts all signatures into a MinHashLSH index and queries for approximate nearest neighbors (Jaccard ≥ 0.5). Any pair of similar flights that receive different predicted labels is flagged as anomalous.

```
lsh = MinHashLSH(threshold=0.5, num_perm=128)
for i, row in enumerate(sample):
m = MinHash(num_perm=128)
for token in [carrier, origin, dest]:
m.update(token.encode())
lsh.insert(f'flight_{i}', m)
# Query each flight for similar neighbors
# Flag if neighbor predicts a DIFFERENT class
anomaly_rate = anomalous_pairs / total_pairs_checked
```

compute_latency_stats() and compute_throughput_stats():

Uses Spark aggregate functions (percentile_approx, stddev, mean) to compute latency statistics across all streaming prediction records. Throughput is computed per batch_id using min/max consumer_ts as the batch window.

Output structure of benchmark_report.json:

```
{
"batch_metrics": { AUC-ROC, F1, precision, recall, accuracy },
"streaming_metrics": { record_count, prediction_distribution },
"batch_metrics_dp_reported": { same fields + dp_epsilon, dp_mechanism },
"streaming_metrics_dp_reported": { ... },
"streaming_latency": { mean_ms, p50_ms, p90_ms, p95_ms, p99_ms },
"streaming_throughput": { mean/max events_per_sec },
"lsh_anomaly_detection": { pairs_checked, anomaly_rate, interpretation }
}
```

SECTION 10

models/benchmark_report.json — Final Results

models/benchmark_report.json Output · Owner: Anees

The final output of the entire pipeline — produced by benchmark.py and used as the source of truth for the course report and paper.

Batch Inference Results:

Metric	Value
Records scored	537,183
AUC-ROC	0.9386
AUC-PR	0.8877
F1	0.9029
Precision	0.9058
Recall	0.9011
Accuracy	0.9011
% Predicted Delayed	23.7%

Streaming Latency (62,815 records):

Metric	Value
Mean latency	5,303 ms
p50 latency	5,212 ms
p90 latency	9,394 ms
p95 latency	9,921 ms
p99 latency	10,438 ms
Min latency	144 ms
Max latency	10,613 ms

LSH Anomaly Detection:

Metric	Value
Sample size	5,000 predictions
Pairs checked	~24.99 million
Anomalous pairs	0

Anomaly rate	0.0%
Interpretation	Predictions are consistent across similar flights

Known Issue — Streaming 0% delayed predictions:

The streaming consumer predicted 0% delayed for all 62,815 streaming records. The most likely cause: ARR_DELAY is a post-flight column that is null in real-time events (the flight hasn't landed yet). The imputer fills it with the training median (a non-delay value), which shifts the decision boundary, causing the model to predict class 0 for every record. Fix: remove ARR_DELAY from features for real-time inference and rely only on pre-departure features (DEP_DELAY, CRS_DEP_TIME, DISTANCE, etc.).
